# Foundations of High Performance Computing

## Lecture 7: network basics for parallel computing

2025-2026 Stefano Cozzini

**"Foundation of HPC" course**

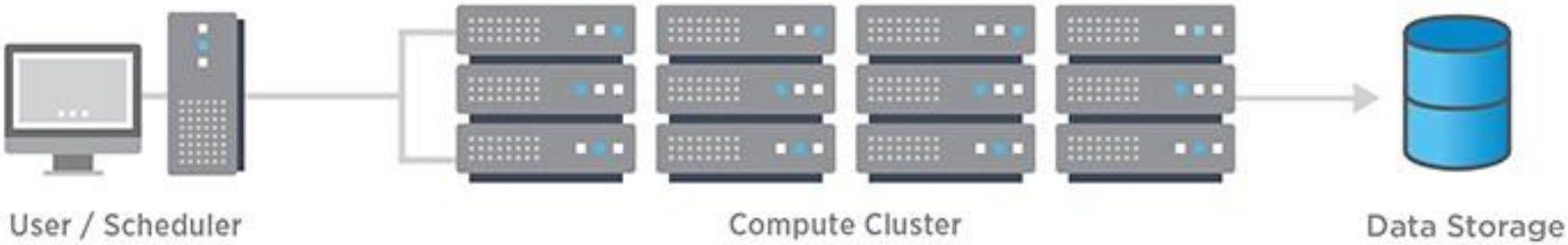
DATA SCIENCE &
SCIENTIFIC COMPUTING

# Agenda

- Network basic for parallel architecture
- Network basic performance characteristics

# Recap on HPC architecture

- Several computers (nodes) often in special cases for easy mounting in a rack
- One or more networks (interconnects) to hook the nodes together
- MP application' performance rely on the characteristics of the networks.



# Network cluster classification

- HIGH SPEED NETWORK
  - parallel computation
  - low latency /high bandwidth
  - Usual choices:  Infiniband...
- I/O NETWORK
  - I/O requests (NFS and/or parallel FS)
  - latency not fundamental/ good bandwidth
  - GIGABIT  could be  ok /10Gb and/or  Infiniband better
- In band Management network
  - management traffic of all services (LRMS/NFS/software etc..)
- Out of band Management network:
  - Remote control of nodes and any other device

# How to model network performance ?

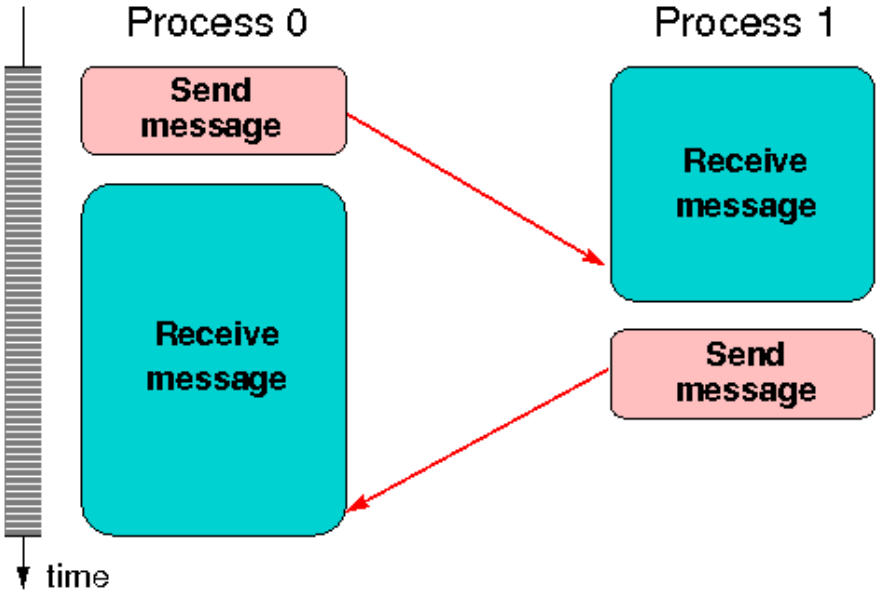
- Network capacity to transfer data
- Very simple model :
  - Total transfer time of a message

$$T_{comm} = \lambda + (Size\ of\ message) / b_{network}$$

  - $\lambda$ is the latency of the network : i.e. the time to setup the communication channel
  - $b_{network}$ is the asymptotic network bandwidth measured in Mb/sec.

# How can we estimate/measure latency and bandwidth ?

- Using a simple “Ping-Pong” program :
  - Two processes on the network exchange point-to-point message.
  - A single message of N Bytes is sent forward and backward: data transfer is 2N



# Ping-Pong algorithm

```
myID= get_process_ID()
if(myID.eq.0) then
  targetID= 1
   S = get_walltime()
   call Send_message(buffer,N,targetID)
   call Receive_message(buffer,N,targetID)
   E = get_walltime()
   MBYTES = 2*N/(E-S)/1.d6 ! MBytes/sec rate
   TIME = (E-S)/(2*1.d6) ! transfer time in microsecs
                         ! for single message
else
  targetID= 0
   call Receive_message(buffer,N,targetID)
   call Send_message(buffer,N,targetID)
endif
```

# Ping Pong implementations

- Available on the most common benchmark suite:

- IMB: Intel MPI benchmark
  - intel/mpi-benchmarks (github.com)

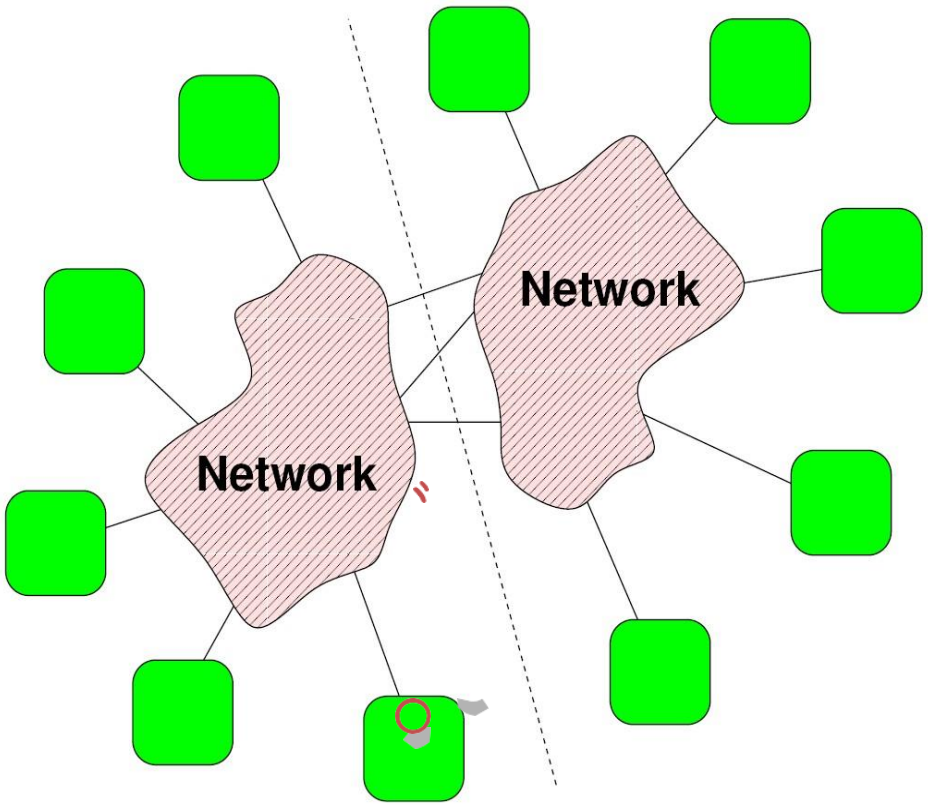
- OSU microbenchmarks
  - MVAPICH :: Benchmarks (ohio-state.edu)

# Network topology

- How the components are connected.
- Important properties
  - Diameter: maximum distance between any two nodes in the network (hop count, or # of links).
  - Nodal degree: how many links connect to each node.
  - Bisection bandwidth: The smallest bandwidth between half of the nodes to another half of the nodes.
- A good topology:
  - small diameter,
  - small nodal degree,
  - large bisection bandwidth

# Bisection bandwidth: $B_b$

- Split N nodes into two groups of N/2 nodes such that the bandwidth between these two groups is minimum
- general metric for the data transfer "capability" of a system
- More meaningful metric in terms of system scalability: $B_b$ /Nodes

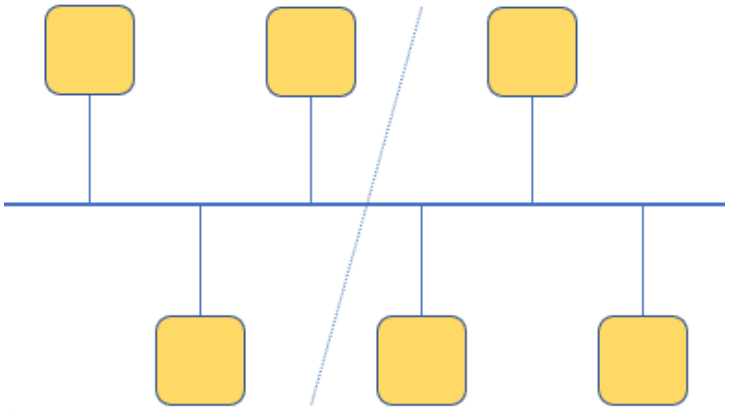


# Common Topologies in HPC

- Bus
- Crossbar switches
- Fat tree
- CBB (Constant Bi-sectional Bandwidth)
- Mesh
  - 3D torus

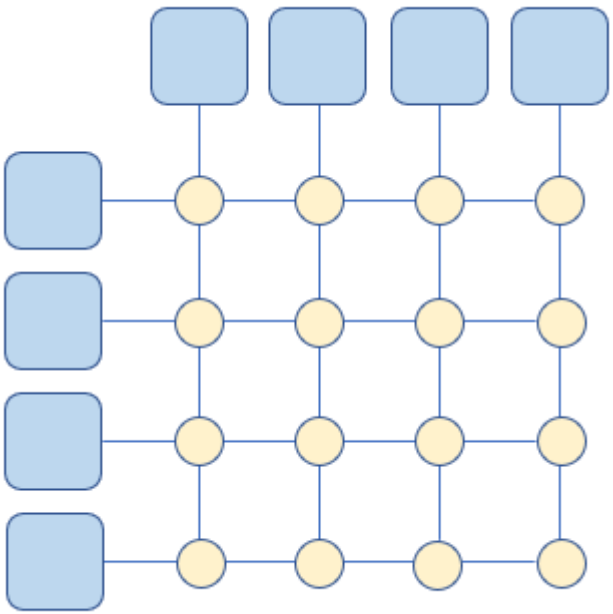
# Bus topology

- Bus can be used by one connection at a time
- Bandwidth is shared among all devices
- Bisection BW is constant: $B_b$/Nnodes ~ 1/Nnodes
- Examples: PCI bus
- Advantages
  - Low latency
  - Easy to implement
- Disadvantages
  - Shared bandwidth, not scalable
  - Problems with failure resiliency (one defective agent may block bus)



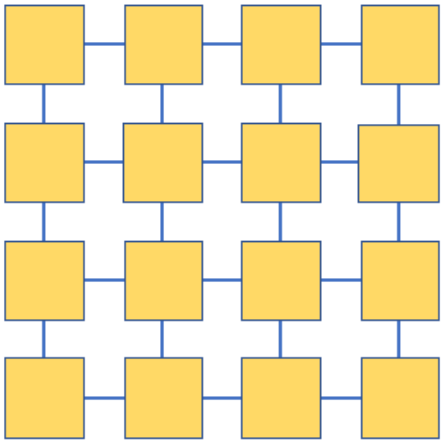
# Non blocking crossbar switch

- A non-blocking crossbar can mediate a number of connections between a group of input and a group of output elements
- This can be used as a 4-port non-blocking switch
- Switches can be cascaded to form hierarchies (common case)
- Allows scalable communication at high hardware/energy costs
- Not feasible for large HPC installations



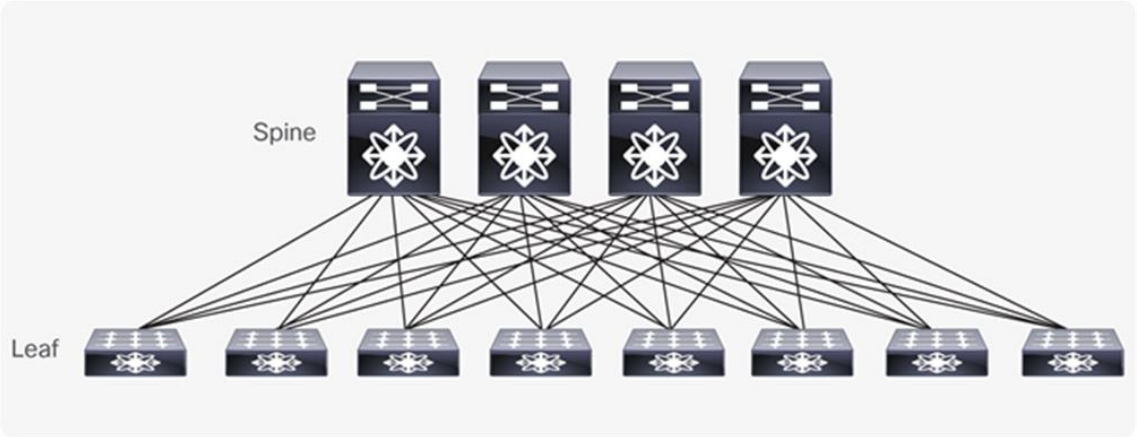
# Meshes

- Fat trees can become prohibitively expensive in large systems
- Compromise: Meshes
  - n-dimensional Hypercubes
  - Toruses (2D / 3D)
  - Many others (including hybrids)
- Each node is a “router”
- Direct connections only between direct neighbors
- Different from a  crossbar!
- Intelligent resource management and routing algorithms are essential



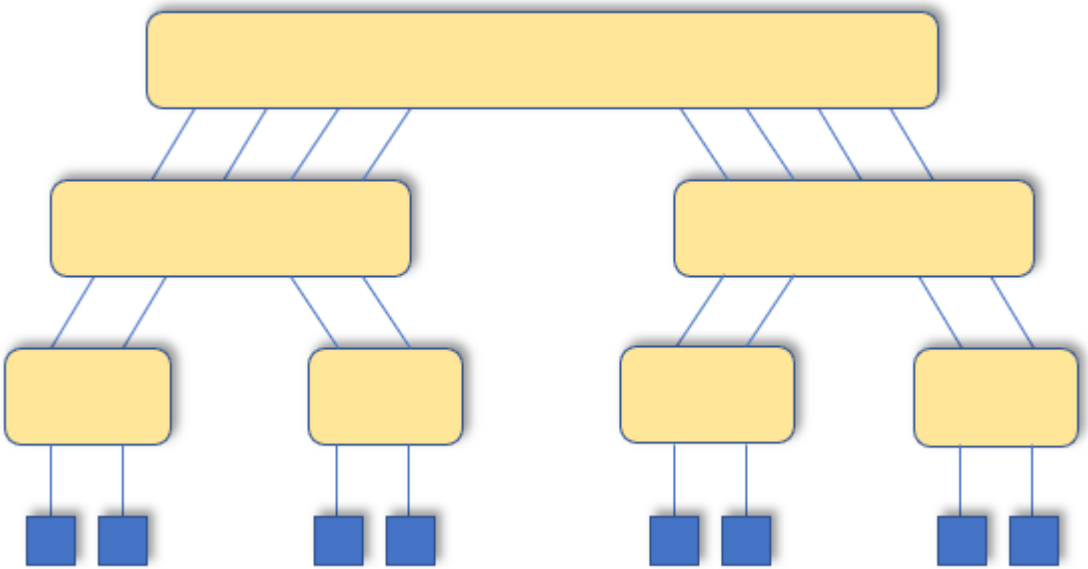
# Switches and Fat-Trees

- HPC clusters are built with switched networks
- Compute nodes (“devices”) are split up in groups – each group is connected to single (non-blocking crossbar-) switch (“leaf/edge switches”)
- Leaf switches are connected with each other using an additional switch hierarchy (“spine switches”) or directly (for small configs.)
- “Perfect” world: fat- trees



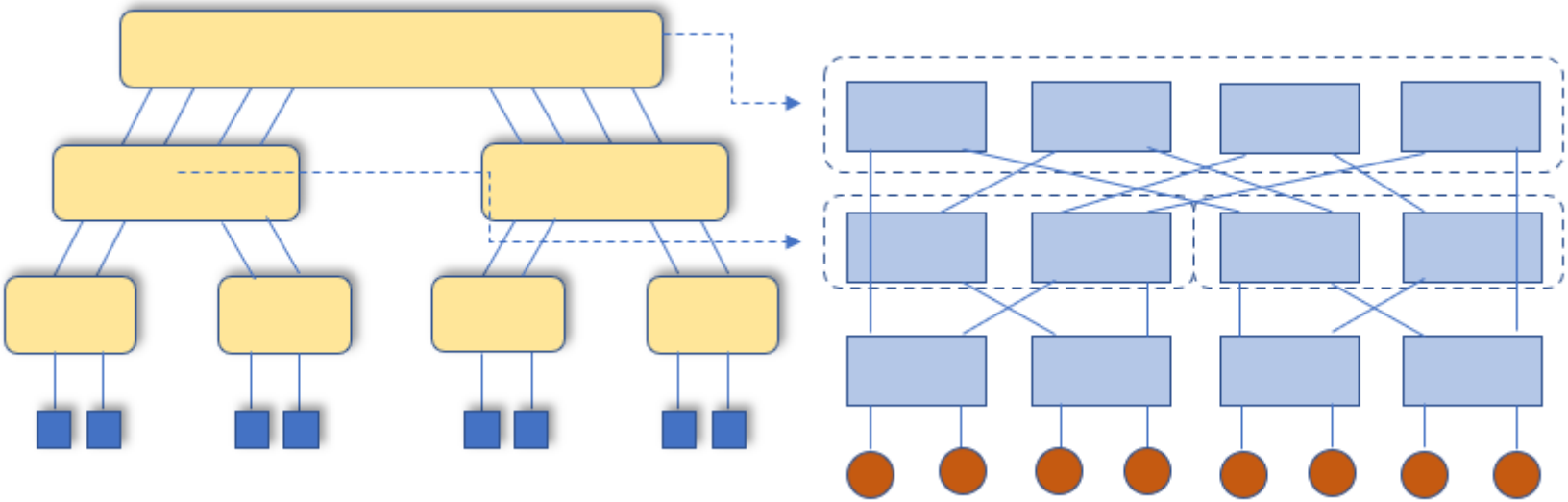
# Fat-trees switch hierarchy..

- Fully non-blocking:
  - Each level double  the number of link of the switches
  - Not practical. Root is NXN switch



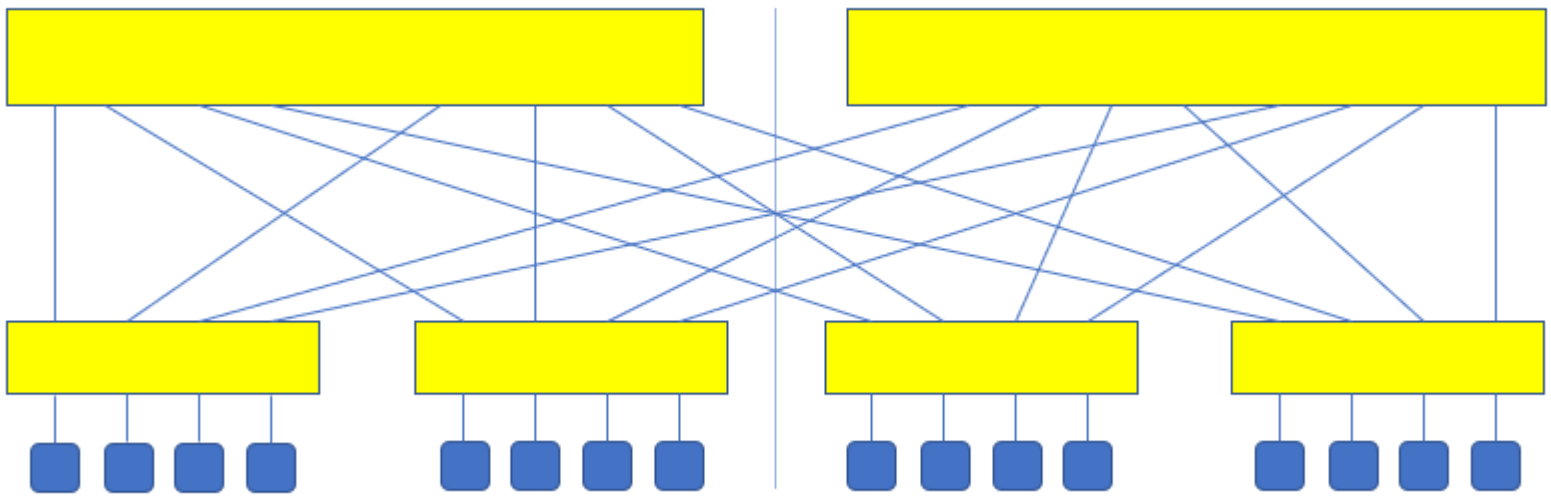
# Practical fat-tree implementation

- Use smaller switches to approximate large switches.
- Most commodity large clusters use this topology.
- Also call constant bisection bandwidth network (CBB)
  - $N_{nodes}/2$ end-to-end connections with full bandwidth
  - $B_b = B * N_{nodes} /2$
  - $B_b / N_{nodes} = \text{const.} = B/2$



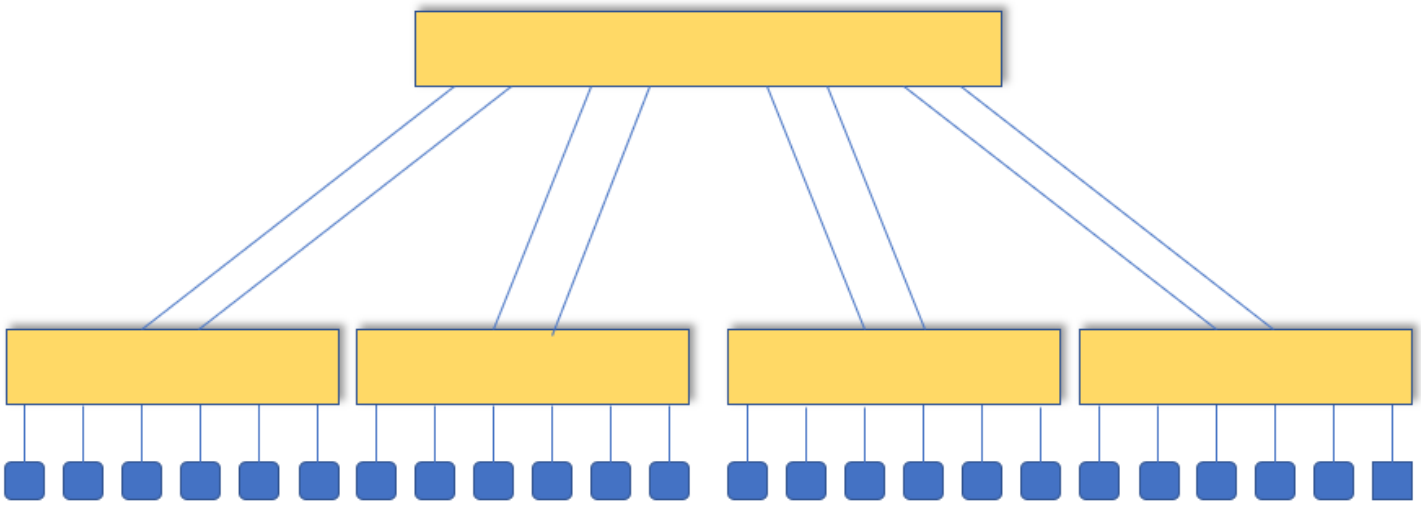
# Two level CBB example

- $N_{nodes}/2$ end-to-end connections with full bandwidth: 8
- $B_b = B * N_{nodes}/2 = 8B$
- $B_b / N_{nodes} = \text{const.} = B/2$



# Oversubscription

- Spine does not support $N_{nodes}/2$ full BW end-to-end connections
- $B_b / N_{nodes}$ = const. = $B/(2k)$, with k oversubscription factor (k=3 for the example)
- Resource management (job placement) is crucial



# High speed networks: what is available ?

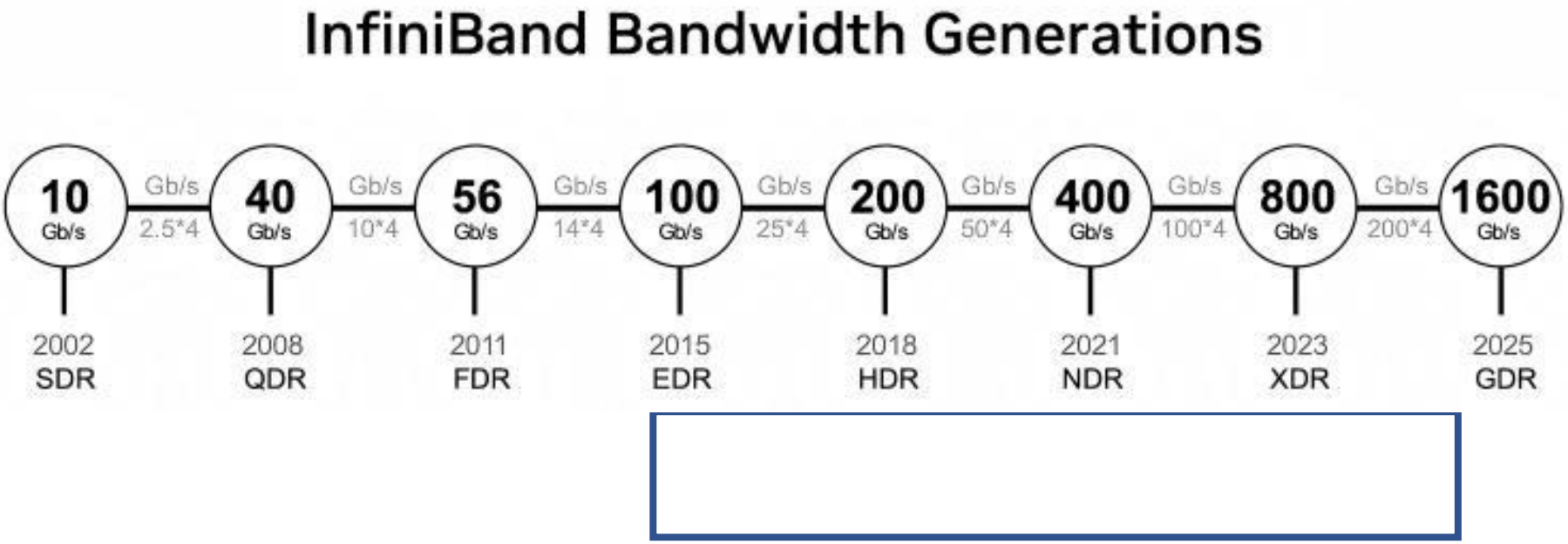
"Big Three" networks currently powering the world's fastest systems are:

- **Cray Slingshot-11:**
  - Developed by HPE, this is the backbone of the world's top exascale systems like **El Capitan** (#1) and **Frontier** (#2). It is highly optimized for congestion management and low latency at massive scales.
- **NVIDIA Mellanox InfiniBand:**
  - This remains the most widely used network across the broader TOP500. It is the standard for GPU-heavy AI clusters, including Italy's **Leonardo** (#7) and Eni's **HPC6** (#5).
- **Slingshot (HPE):**
  - Also used in major European systems like **LUMI** (#8), providing a bridge between traditional Ethernet and specialized HPC networking.
- All of them are used behind your MPI implementation..

# Infiniband speed: physical layer..

- InfiniBand uses serial stream of bits for data transfer
- Linkwidth
  - 1x – One differential pair per Tx/Rx
  - 4x – Four differential pairs per Tx/Rx
  - 12x - Twelve differential pairs per Tx and per Rx
- LinkSpeed
  - Single Data Rate (SDR) - 2.5Gb/s per lane (10Gb/s for 4x)
  - Double Data Rate (DDR) - 5Gb/s per lane (20Gb/s for 4x)
  - Quad Data Rate (QDR) - 10Gb/s per lane (40Gb/s for 4x)
  - Fourteen Data Rate (FDR) - 14Gb/s per lane (56Gb/s for 4x)
  - Enhanced Data rate (EDR) - 25Gb/s per lane (100Gb/s for 4x)
  - High Data rate (HDR). - 50Gb/s per lane (200Gb/s for 4x)
  - ….
- Linkrate
  - Multiplication of the link width and link speed
  - Most common shipping today is 4x ports EDR/NDR

# InfiniBand performance over time



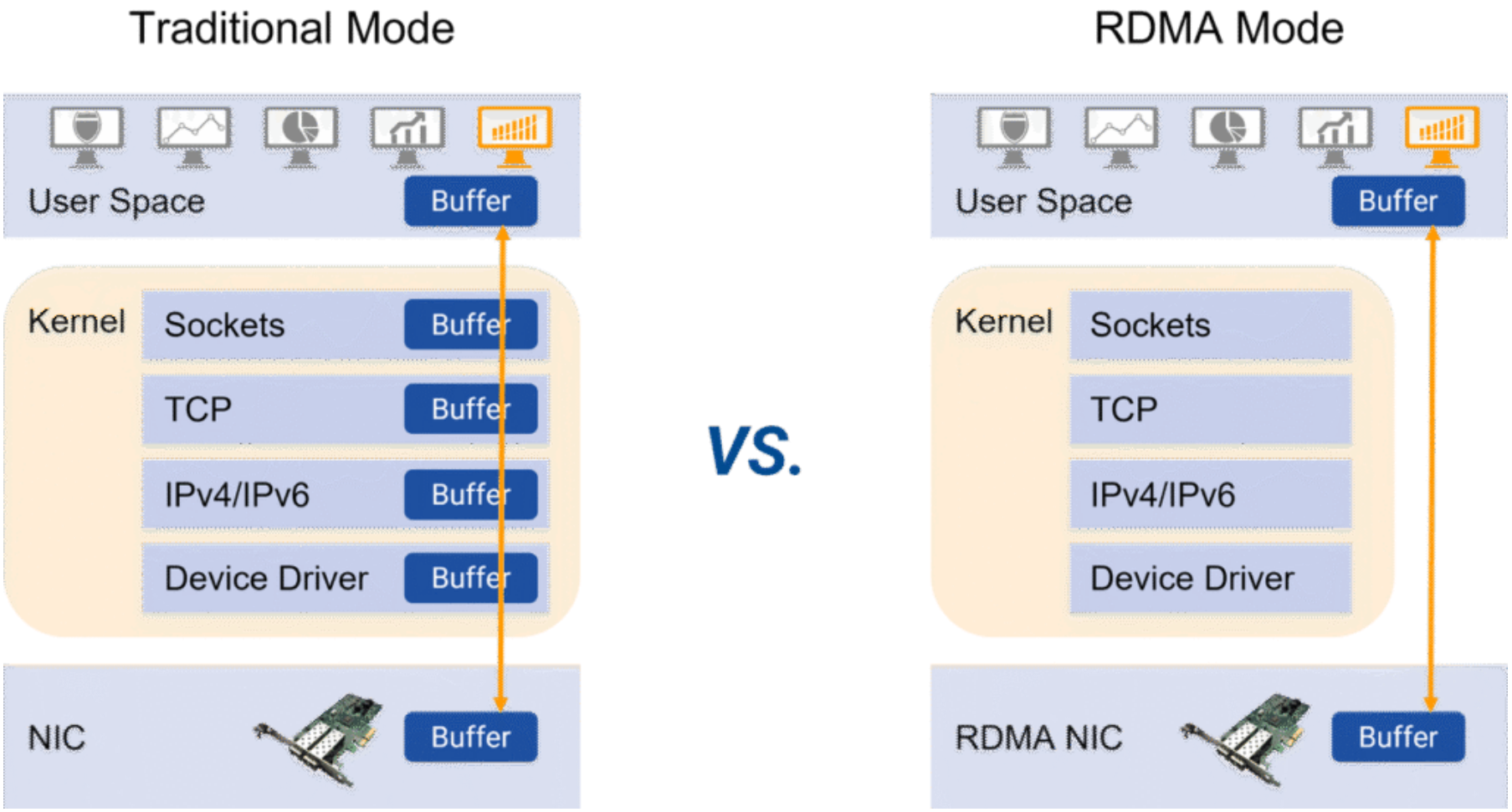
# A short review: the past

- **SDR (2001)**
  - SDR (Single Data Rate( utilizing 8b/10b encoding with a per-lane rate of 2.5 Gbit/s.
  - A 4x link delivers a theoretical throughput of 10 Gbit/s, with an actual rate of approximately 8 Gbit/s.
- **DDR (2005)**
  - DDR (Double Data Rate) is an InfiniBand standard introduced in 2005, also employing 8b/10b encoding but doubling the per-lane rate to 5 Gbit/s. A 4x link achieves a theoretical throughput of 20 Gbit/s, with an actual rate of around 16 Gbit/s. It similarly relies on CX4 connectors.
- **QDR (2007)**
  - QDR (Quadruple Data Rate) retaining 8b/10b encoding while increasing the per-lane rate to 10 Gbit/s. A 4x link offers a theoretical throughput of 40 Gbit/s, with an actual rate of roughly 32 Gbit/s.
- **FDR (2011)**
  - FDR (Fourteen Data Rate), adopting the more efficient 64b/66b encoding with a per-lane rate of 14.0625 Gbit/s. A 4x link provides a theoretical throughput of 56 Gbit/s, with an actual rate of approximately 54.54 Gbit/s.

# A short review : present and future

- **EDR (2014)**
  - EDR (Enhanced Data Rate utilizing 64b/66b encoding with a per-lane rate of 25.78125 Gbit/s. A 4x link delivers a theoretical throughput of 103.125 Gbit/s, with an actual rate of approximately 100 Gbit/s.
- **HDR (2018)**
  - HDR (High Data Rate) employing 256b/257b encoding with a per-lane rate of 53.125 Gbit/s. A 4x link provides a theoretical throughput of 212.5 Gbit/s, with an actual rate of around 200 Gbit/s.
- **NDR (2022)**
  - NDR (Next Data Rate), employing 256b/257b encoding with a per-lane rate of 106.25 Gbit/s. A 4x link achieves a theoretical throughput of 425 Gbit/s, with an actual rate of approximately 400 Gbit/s.
- **XDR (2024)**
  - XDR (eXtreme Data Rate), utilizing 256b/257b encoding with a per-lane rate of 212.5 Gbit/s. A 4x link delivers a theoretical throughput of 850 Gbit/s, with an actual rate of approximately 800 Gbit/s.
- **GDR (2025-2028)**
  - GDR (Gxx Data Rate) is the next-generation InfiniBand standard, targeting double the speed of XDR. With XDR products commercially launched in 2025, GDR is expected to finalize its specification in 2026 and achieve commercial availability by 2028.

# Infiniband vs Ethernet..



# Orfeo network classification

- HIGH SPEED NETWORK
- I/O NETWORK

100/200 Gbit HDR Infiniband

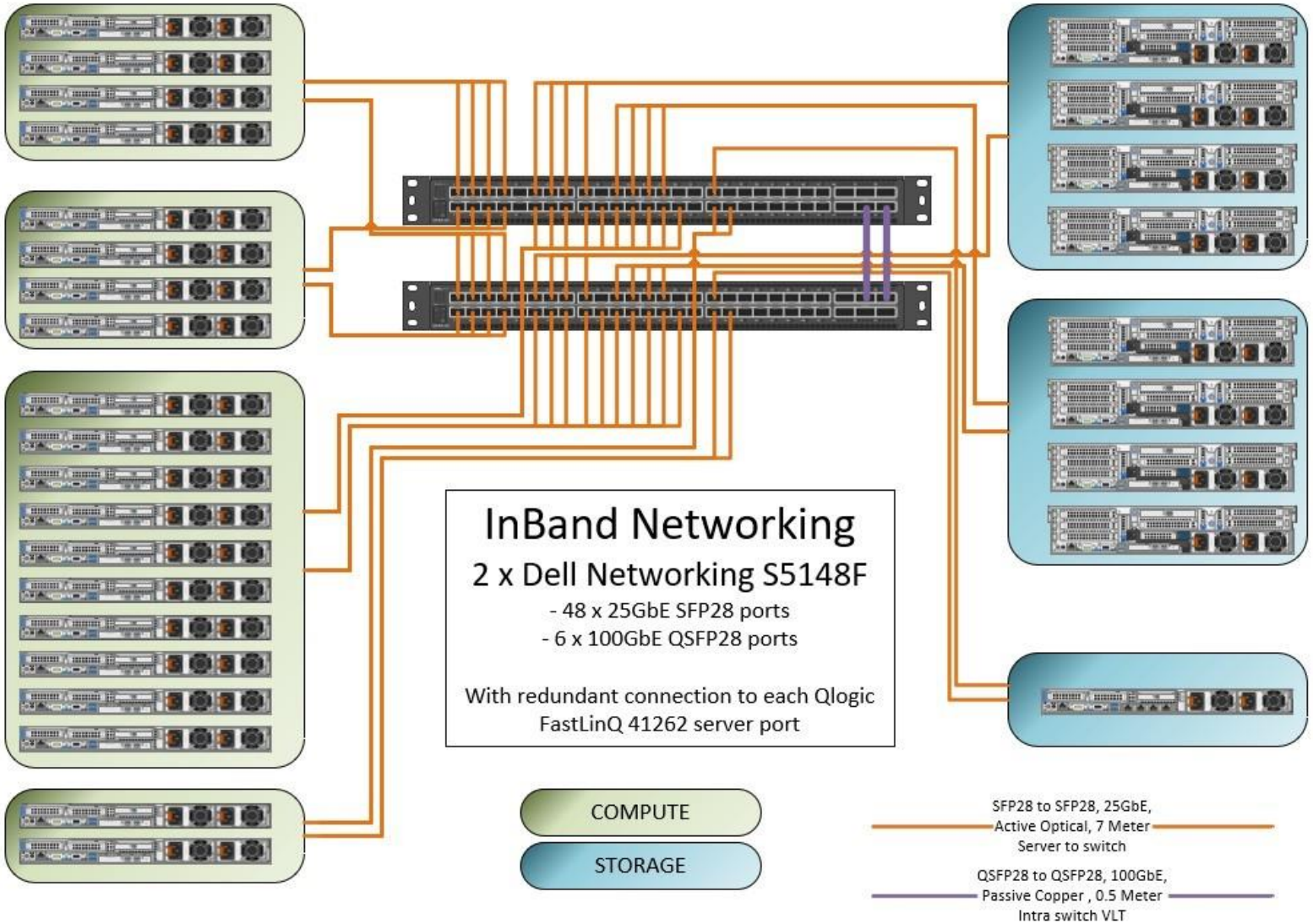
- In band Management network

25Gbit Ethernet

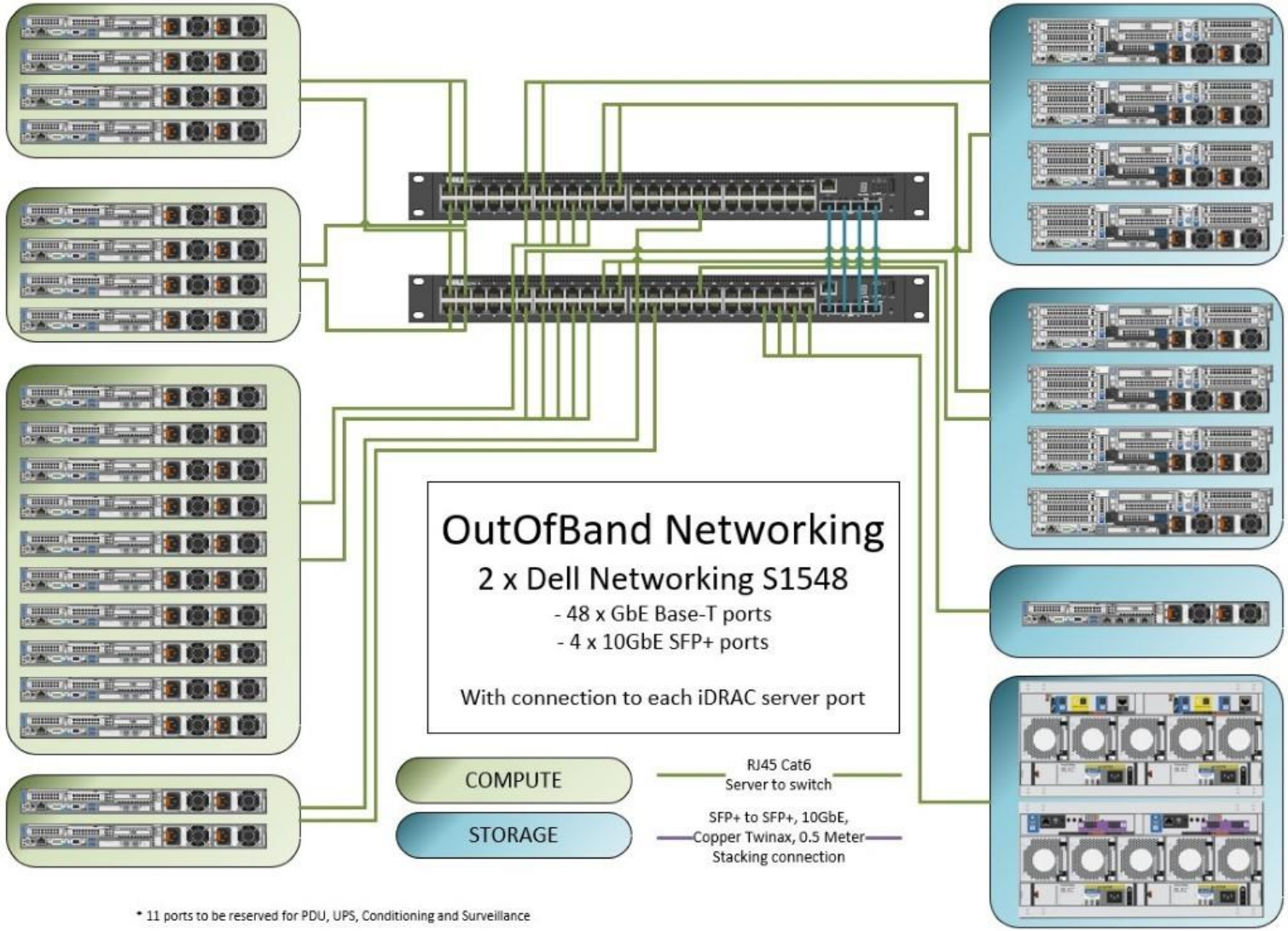
- Out of band Management network:

1Gbit Ethernet

# Orfeo in band management network: 25 Gbit ethernet

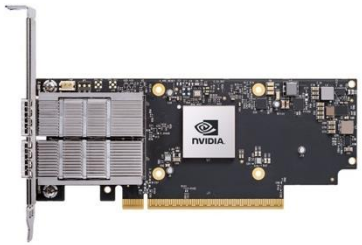


# Orfeo out of band management network: 1 Gbit ethernet

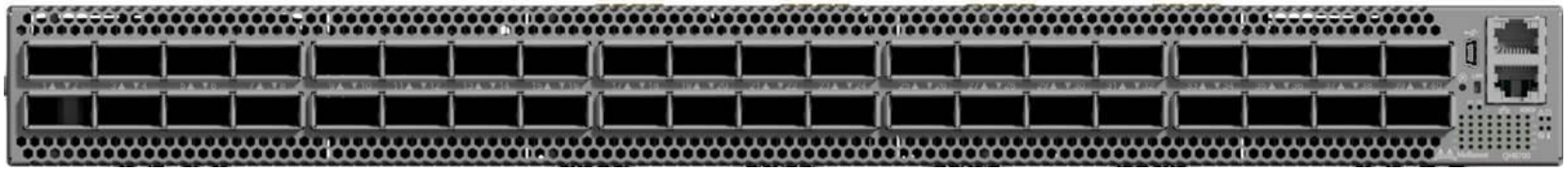


# Orfeo High Speed network:

- 2 switches:
  - Q9700 : 40 ports with 400 Gbit/sec
  - Q8700 : 40 ports with 200 Gbit/sec
- tens of network cards:
  - Connect X6 100Gbit/sec
  - Connect  X7  200Gbit/sec
- Tens of splitting cables (optical /copper)
  - 2 to 1 ( 2 x 100 to 200 )
  - 4 to 1 ( 4 x 100 to 400)
  - 2 to 1 ( 2 x 200 to 400)



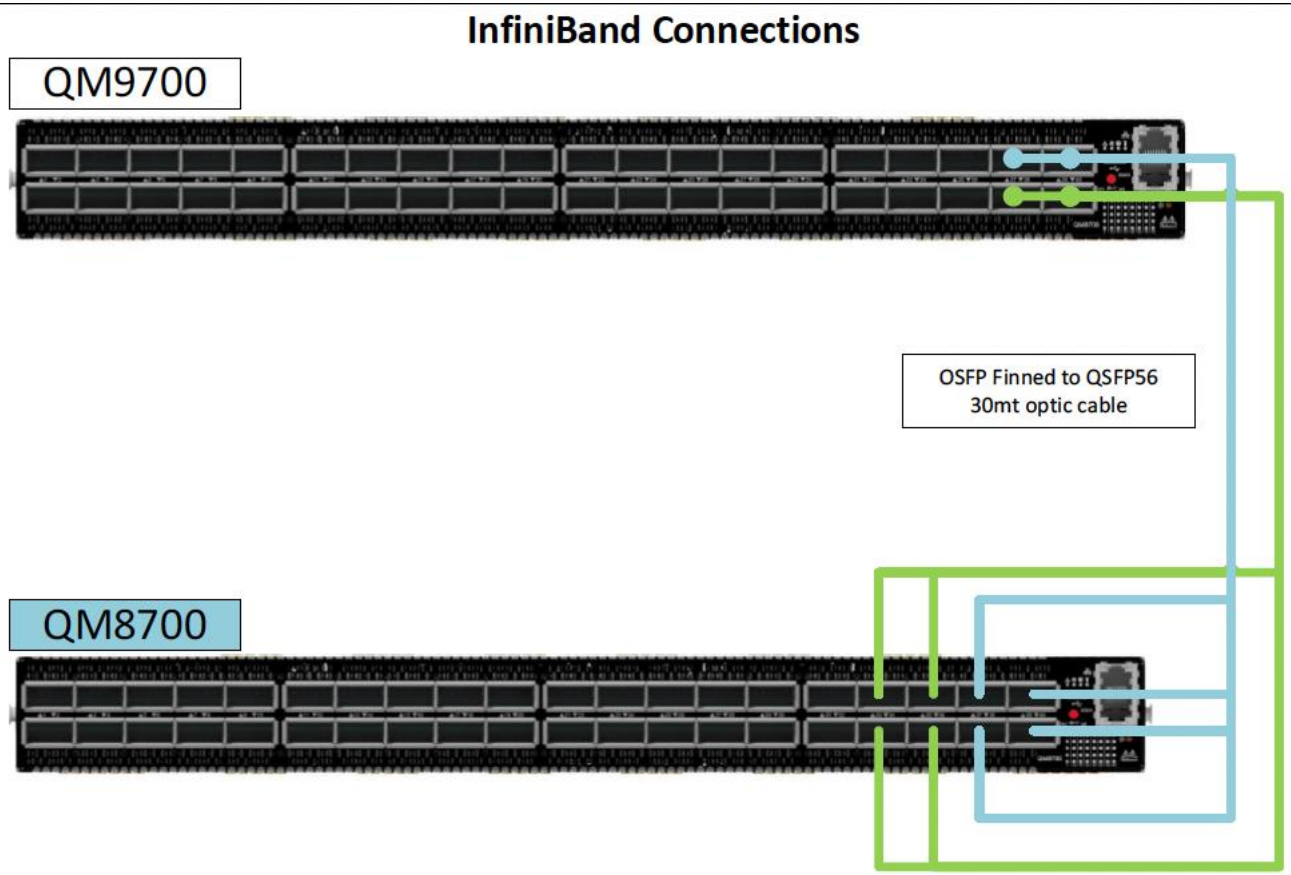
# ORFEO IB network



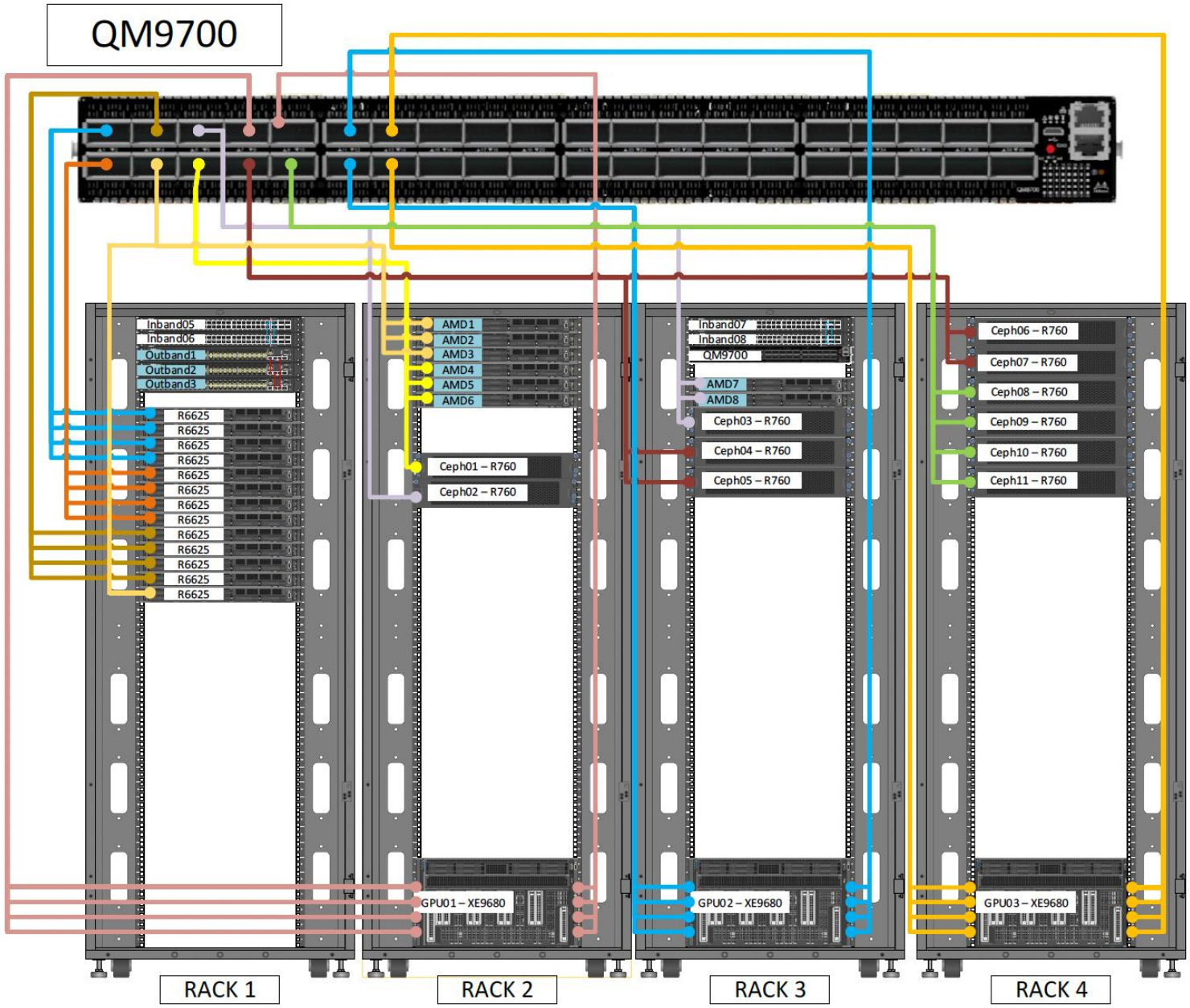
Performance

- 40 x HDR 200Gb/s ports in a 1U switch
- 80 x HDR100 100Gb/s ports (using splitter cables)
- 16Tb/s aggregate switch throughput
- Sub-130ns switch latency

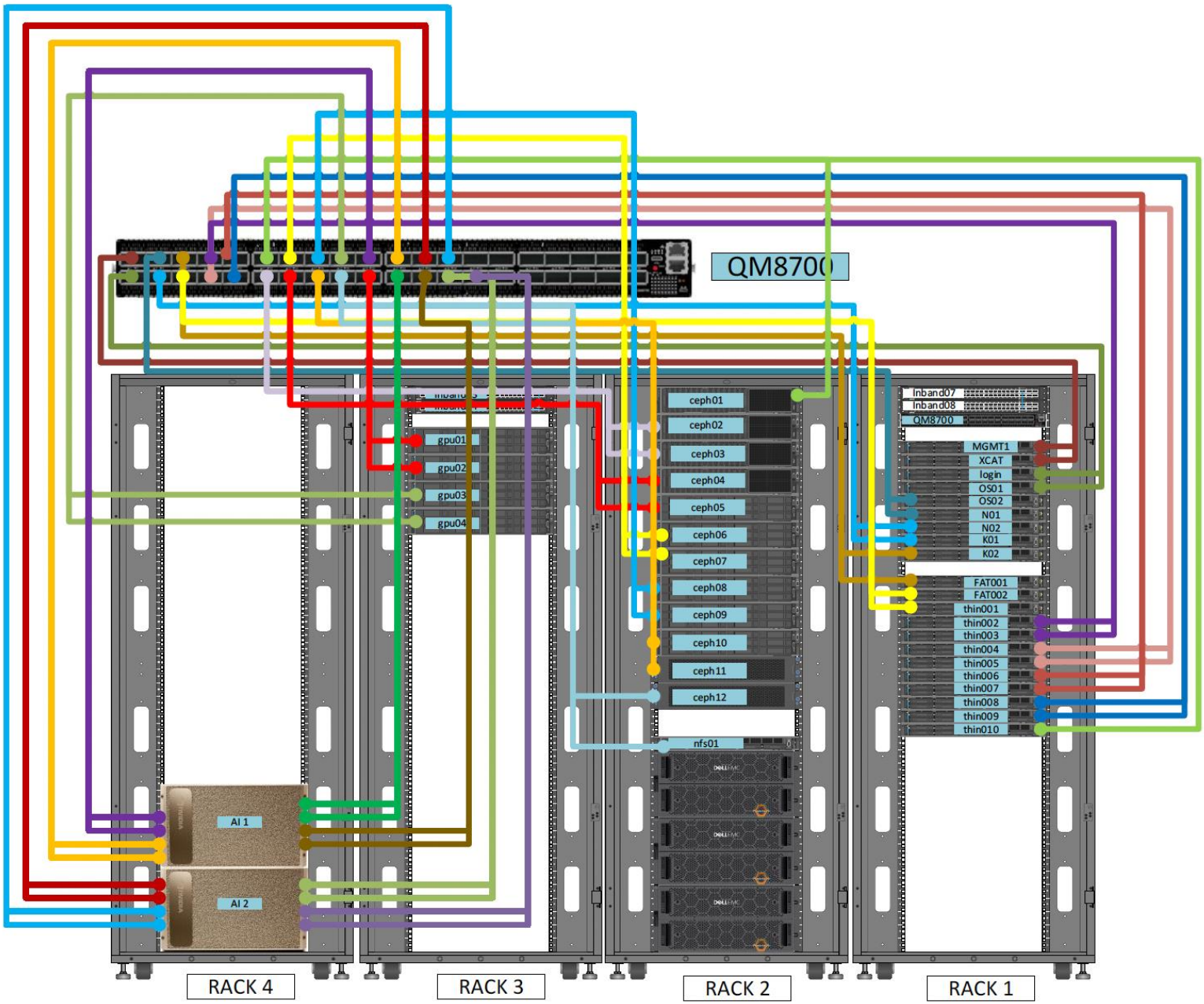
# Infiniband switch interconnection: 400 Gbit



# Array 1 Infinibad Cabling



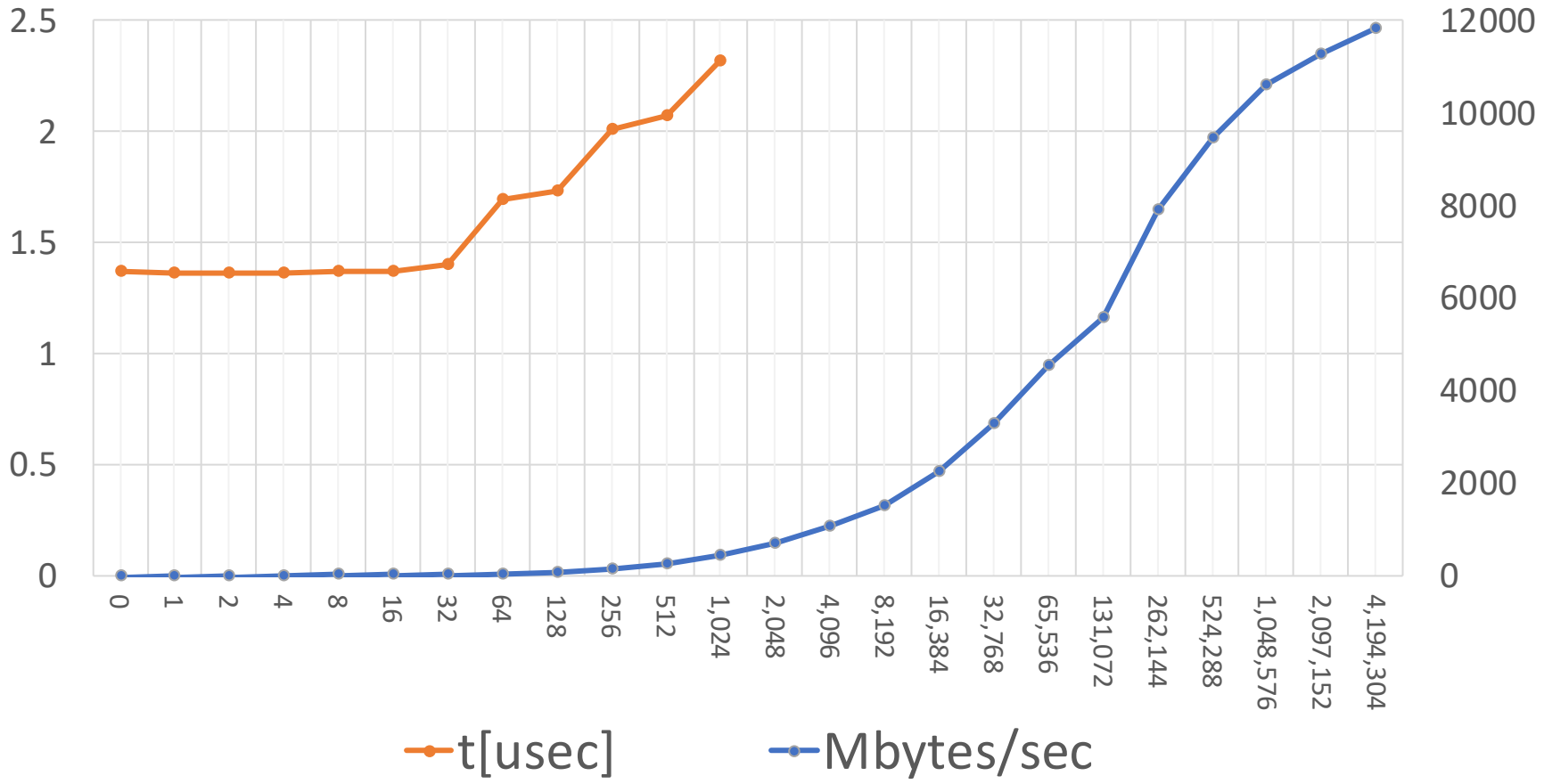
## Array 2 Infinibad Cabling



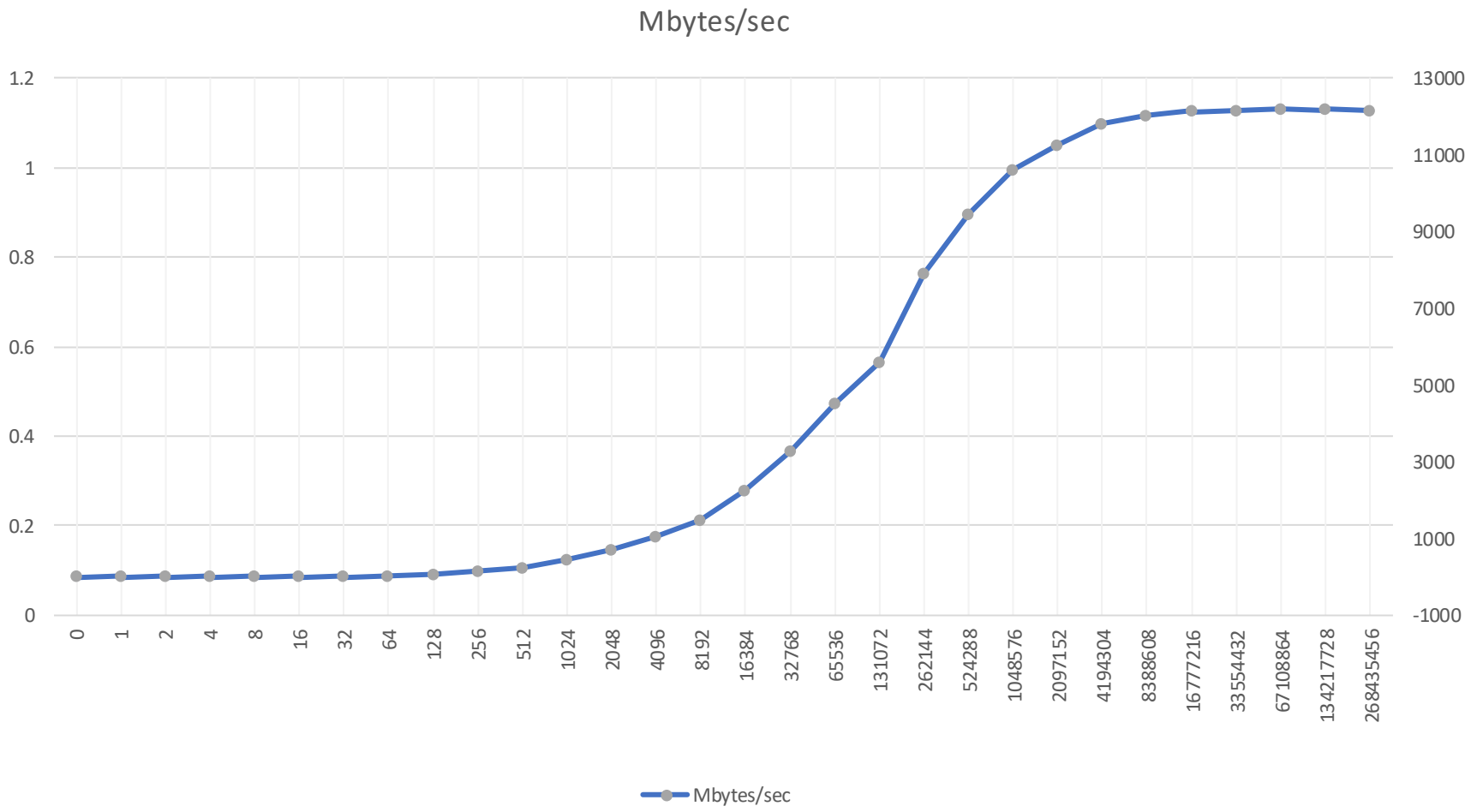
# Measuring MPI point-to-point performance on Orfeo

- Download Intel MPI benchmark
- Compile it
- Run it
- Get results and interpret it

# Measuring MPI point-to-point performance on Orfeo



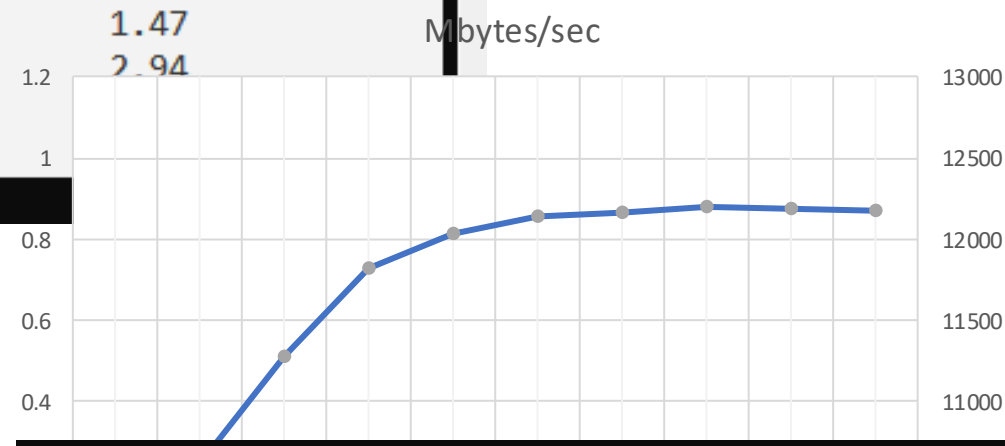
# Measuring MPI performance on Orfeo



# Extrapolating values for internode communication

```
#---------------------------------------------------
# Benchmarking PingPong
# #processes = 2
#---------------------------------------------------
       #bytes #repetitions      t[usec]   Mbytes/sec
            0         1000         1.36         0.00
            1         1000         1.36         0.74
            2         1000         1.36         1.47
            4         1000         1.36         2.94
            8         1000         1.36
           16         1000         1.36
           32         1000         1.40
```

$\lambda$




$b_{network}$

| | | | |
|---|---|---|---|
| 4194304 | 10 | 355.03 | 11813.78 |
| 8388608 | 5 | 697.14 | 12032.92 |
| 16777216 | 2 | 1382.15 | 12138.48 |
| 33554432 | 1 | 2757.62 | 12167.91 |
| 67108864 | 1 | 5500.71 | 12200.04 |
| 134217728 | 1 | 11012.36 | 12187.92 |
| 268435456 | 1 | 22054.87 | 12171.25 |

# Our network: ORFEO ones..

- We can assume full non-blocking network:

  P/2 pair of nodes communicate in parallel at full speed

$$T_{comm} = \lambda + \text{message-size}/b_{network}$$

- Where

  $\lambda = 1.35 \text{ microsecond}$

  $b_{network} = 12\text{Gb/second}$

# Exercise/Tutorial:

- Confirm the data of the previous slide for all the kind of nodes and all the kind of MPI libraries.